

DataSyncX ETL Agent Playbook

This playbook is the durable knowledge base for the repository-local `DataSyncX ETL Agent`.

Use the agent as the default first choice when the task is any of the following:

- write or review an ETL script that may use DataSyncX,
- decide whether an ETL behavior belongs in a product repository or in DataSyncX,
- design batching, query parameterization, task splitting, or runtime metadata flow,
- evaluate the minimum safe validation gate for an ETL change.

The agent should stay thin. Put stable ETL-routing rules, evidence order, and DataSyncX-specific operating guidance here so the agent can read and apply them instead of carrying all detail in the prompt body.

Default ETL Entry Rule

When an ETL-writing request could reasonably use DataSyncX, prefer the `DataSyncX ETL Agent` over a generic coding agent.

This rule is about decision quality, not repository ownership. The agent may conclude that the final patch belongs in a product repository, but it should still be the first ETL reasoning surface because it knows the framework abstractions, reuse boundaries, and validation expectations.

Evidence Stack

Before recommending implementation, inspect repository evidence in this order:

1. `docs/agent-rules-version.md` as the lightweight freshness probe.
2. This playbook for routing and validation discipline.
3. User-facing docs for the closest workflow shape.
4. Maintained demos for the nearest runnable baseline.
5. Runtime code and adjacent tests for current behavior.

Start with the narrowest useful source:

- `docs/usage.md` for workflow choice: free-style, direct demo copy, or demo factory.
- `docs/readers.md`, `docs/writers.md`, and `docs/processors.md` for component capabilities.

- `docs/concepts.md` and `docs/api/index.md` for orchestration concepts and API details.
- `src/datasyncx/demo/` for copy-first examples that reflect current supported patterns.

If docs and code disagree, report the mismatch explicitly and trust current code plus validation over stale prose.

Rule Version Probe (Low-Cost Auto Refresh)

Use the version marker to keep behavior current across restored sessions without forcing full-doc rereads every turn.

1. Read `docs/agent-rules-version.md` at session start and when a previously saved chat is resumed.
2. Compare marker values to the currently active tuple in chat context:
 - `agent_rules_version`
 - `playbook_rules_version`
 - `copilot_guidance_version`
3. Reload full guidance only when the tuple changed or no tuple is active yet:
 - `docs/etl-agent-playbook.md`
 - `docs/copilot-agent.md`
4. If tuple is unchanged, skip full-doc reload and continue normal compact dialogue.
5. Do not ask the user to manually request a reread when marker values already show drift.

DataSyncX-First Decision Rules

Prefer existing DataSyncX abstractions before inventing local framework-like helpers.

Check these levers first:

- `Task` and `TaskExecutor` for orchestration.
- `TaskRangeBuilder` variants for batching, windowing, and split execution.
- `QueryBuilder` or job-spec metadata for parameterized SQL.
- Existing readers, writers, and processors before custom wrappers.
- Direct demo copy or demo-factory generation before creating a new script shape from scratch.

Keep the change product-local when the behavior is tied to one product's source schema, target schema, predicate, schedule, credential flow, or business rule.

Move the change into DataSyncX only when the capability is broadly reusable and the repository can support it with focused tests plus adjacent regression coverage.

Minimum Working Procedure

For each ETL task, do this in order:

1. Identify the active repository, branch, runtime path, and current local changes.
2. Freeze the validation baseline: active slice, watermark, sample input, or expected runtime shape.
3. Classify the task as product-local, framework-worthy, documentation-only, or validation-only.
4. Find the nearest existing DataSyncX pattern before proposing new structure.
5. Choose one bounded candidate path.
6. Make the smallest safe edit set.
7. Run the narrowest validation that can falsify the approach.
8. Report residual risk and the next exact validation step if any environment-gated checks were skipped.

Deterministic Interaction Flow

Use this flow for user-facing execution:

1. Generate

- Produce one bounded script or template path quickly.
- Do not block script creation on non-critical details.

1. Patch

- Ask one decision per turn and patch incrementally.
- Prioritize high-impact correctness fields first.

1. Validate

- Keep manual-first command guidance as default.
- Interpret user-run outputs and propose next minimal patch.

1. Optional Deploy

- Only after local behavior is accepted.
- Enforce version-alignment checks for image-based deployment guidance.

Routing State Rules

Use these route rules in order.

1. S0 Intake

- If source-target is missing, ask only the minimum fields needed to choose a path.

1. S1 Capability Lookup

- Demo first, then demo_factory support, then component-level support.

1. S2 Path Selection

- Direct demo copy when a maintained example exists.
- Treat direct demo as exact-copy behavior on the first turn.
- Exact copy means source demo content is unchanged; only destination path and filename differ.
- Do not alter defaults, imports, SQL, placeholders, credential handling, process_type, or comments in first-turn direct demo output.
- Apply any change only in later patch-loop turns with explicit user confirmation.
- Demo factory when template-driven generation is supported.
- Free-style scaffold when user asks for custom composition or template coverage is absent.
- Unsupported escalation when requested capability is not present.

For known source-target pairs with an exact maintained demo:

- Use fast path: perform one direct copy action and avoid broad exploratory narration.
- Return compact response only: copied artifact, one-line validation gate, one next question.

Environment And Install Preflight

Before claiming a generated script is run-ready, check environment and install readiness.

1. Target venv readiness

- Confirm the script target project and its active venv.
- Confirm whether `datasyncx` is installed in that same venv.

1. Windows TMG PyPI readiness

- If installation or upgrade is needed on Windows, check whether `C:\ProgramData\pip\pip.ini` exists and includes expected `trusted-host` and `index-url` values.

1. Permission boundary

- Treat system-wide pip config guidance as permissioned.
- Ask for user approval before suggesting or applying system-level configuration steps.

1. Installation and verification

- Guide install or upgrade in the target project venv.
- Verify with `python -c "import datasyncx; print(datasyncx.__version__)"` in that same venv.

Free-Style Performance Advisory

For free-style path, provide optional topology guidance with dual triggers.

1. Proactive trigger

- Always include one lightweight recommendation for multi-thread batch read plus bounded single-queue write.

1. Reactive trigger

- Escalate guidance when symptoms appear: queue-depth pressure, writer latency dominance, connection churn, or unstable throughput.

1. Safe defaults

- Moderate queue maxsize.
- Flush rows aligned with writer chunk behavior.
- Bounded retry and deterministic worker shutdown.

1. Boundary

- Keep framework-level guidance generic.
- Keep product business semantics in product repos.

Unsupported Capability Procedure

When capability is not supported:

1. Mark unsupported explicitly.
2. Ask user approval before creating a capability issue.
3. If issue workflow fails, provide fallback contact path to damon.tang@intel.com with concise request payload.
4. Offer a minimal writer bootstrap checklist while keeping the current script path bounded.

Prompt-Completion Discipline

If the user asks for code with a short prompt such as `write Xeus to Postgres`, do not stop after generating a generic script.

Always separate:

- Safe placeholders: fields that can remain temporarily generic without changing control flow.
- Must-confirm items: fields that affect correctness, write semantics, batching, or deployment behavior.

The must-confirm list usually includes:

- target table or schema,
- `process_type`,
- merge or business keys,
- batching or range-builder shape,
- runtime parameters,
- credential path,
- validation target and environment.

If the prompt is underspecified, return the safest bounded baseline and name exactly what still must be confirmed before production use.

For short source-to-target asks, the first response should be concise and action-oriented:

1. confirm route,
2. ask output path,
3. generate bounded scaffold/copy result,
4. ask next highest-leverage single question.

First-Reply Templates

Use these templates to keep first responses short and deterministic.

1. Source-target provided

- Route chosen: <direct demo | demo_factory | free-style>.
- Next action: provide output path.
- Immediate delivery: copy or scaffold one bounded artifact.
- Next single question: ask the highest-leverage missing field.

For direct demo route specifically:

- Copy the nearest maintained demo first.
- Do not change any file content on the first turn.
- Move any behavior changes into the later patch loop after user confirmation.

1. Source-target missing

- Ask only source type and target type first.
- Offer mode choices: direct demo, demo_factory, free-style.
- Do not generate speculative script content before route selection.

1. Unsupported capability

- State unsupported explicitly.
- Ask user approval for capability-issue escalation.
- Provide fallback contact path and minimal writer bootstrap direction.

Concise Dialogue Rule

Default to concise guided responses for short asks.

1. No process narration

- Do not output internal progress narration in user-facing replies.

1. Compact default response

- Keep default reply to:
 1. route plus artifact,
 2. one-line validation gate,
 3. one next question.

1. Expanded response only when asked

- Use full multi-point decision/risk output only when the user explicitly asks for detailed rationale, review-style analysis, or risk breakdown.

One-Question Patch Prompt Bank

Use these prompts in order and ask only one per turn.

1. Range mode

- “Which range mode should we apply: time, token, or none?”

If user selects time mode, use runtime-parameter wording (not vague custom wording):

- “DataSyncX time range currently supports two main input modes:”
 1. “relative time expressions, such as `current_time - 12h`, `current_time - 2d`, or date-only `current_date - 7d`”
 2. “absolute start/end values passed in by the caller or scheduler”

- “For absolute values, supported granularity depends on `date_format`, and common examples are: day (`2026-06-17`), hour (`2026-06-17 08`), and minute (`2026-06-17 08:30`)”
- “Split size is controlled separately by `freq`, for example `1d`, `2h`, or `30m`”
- “Which mode do you want for this script: relative time expression or absolute start/end values passed in each run?”
- “If `QueryBuilder(..., timezone=True)` is used, treat runtime values as UTC-facing inputs. In current DataSyncX behavior, relative `current_time ...` expressions are converted to UTC first, and query generation adds the timezone offset placeholder when building SQL.”
- Do not ask weaker variants such as “keep defaults or specify fixed dates” because that hides the supported time-range model.

1. SQL placeholder alignment

- “Please share SQL so I can align placeholders for the chosen range mode.”

1. Writer mode

- “Should target write mode be append, merge, or replace?”

1. Merge keys (conditional)

- “Merge mode selected. Which merge keys should be used?”

1. Credential source

- “Should credentials come from `get_config_value/env` path or an existing auto-resolution flow?”

1. Runtime basics

- “Confirm minimum runtime values: target server, database, table, and schema if not default.”

Acceptance Scenario Matrix

Use this matrix before calling the interaction upgrade stable.

1. Demo happy path

- Input: user asks common supported source-target pair.
- Expected: route to maintained demo copy path first.

1. Factory happy path

- Input: user requests template-driven generation.
- Expected: route to demo_factory and produce one bounded config/script result.

1. Free-style proactive advisory

- Input: user requests free-style script generation.
- Expected: scaffold generated quickly and lightweight optional performance advisory included.

1. Performance escalation

- Input: user reports throughput or queue pressure symptoms.
- Expected: stronger queue/flush/retry tuning guidance appears.

1. Unsupported capability

- Input: user asks unsupported reader/writer/pair.
- Expected: approval-based escalation plus fallback contact path and minimal bootstrap advice.

1. Deployment handoff

- Input: user asks for container deployment after local acceptance.
- Expected: version-alignment check and bounded Docker steps.

Validation Rules

Treat each implementation as a controlled change with scope, risk level, validation evidence, docs impact, and rollback note.

For runtime changes under `src/datasyncx/**`:

- run targeted tests for the touched module,
- run adjacent regression tests for the same public behavior,
- run an integration slice when one exists and the environment is available.

If integration prerequisites are missing, report `skipped` and name the missing prerequisites explicitly.

For docs or agent-customization changes:

- validate markdown/frontmatter shape,
- validate links and docs navigation,
- update `docs/changelog.md`.

Safety Boundaries

Do not:

- blur product-repository commits and DataSyncX commits,
- claim behavior preservation without a fixed validation baseline,
- introduce inline credentials or insecure defaults,
- add import-time I/O or network effects,
- swallow exceptions broadly without a bounded reason.

In generated or reviewed DataSyncX code, treat plaintext password literals as defects. Prefer `get_config_value(...)` or an existing reader/writer auto-resolution path when the component already derives credentials from the configured account.

Output Contract

Default compact shape:

1. Route and artifact created or selected
2. Minimum validation gate (one line)
3. Next exact question or action needed to continue the patch loop

Expanded shape (only on explicit request):

1. Current ETL lane
2. What the repository evidence currently says
3. Next best bounded candidate path
4. Why this should be DataSyncX-first or product-local
5. Repository-routing decision
6. Minimum validation gate
7. Missing inputs or production-critical parameters still to confirm
8. Risk and rollback note
9. What alternative path is explicitly not chosen now
10. Next exact question or action needed to continue the patch loop